

06d — CSP (Content Security Policy)

Key Concepts

- **CSP** — HTTP response header that tells the browser which content sources are trusted
 - **XSS (Cross-Site Scripting)** — attack where malicious scripts are injected into a page; CSP's primary target
 - **Inline script** — `<script>` tag or `onclick=` directly in HTML; CSP can block these entirely
 - **Nonce** — unique token per request that allows specific inline scripts safely
 - **Report-only mode** — CSP violations logged but not blocked; used for safe rollout
-

How It Works

Server response header:
Content-Security-Policy: script-src 'self' https://cdn.myapp.com

Browser behaviour:

- ☑ Script from myapp.com → allowed
- ☑ Script from cdn.myapp.com → allowed
- ✗ Script from evil.com → blocked
- ✗ Inline `<script>` tag → blocked (unless nonce used)

Common Directives

Directive	Controls
<code>script-src</code>	Where JavaScript can load from
<code>style-src</code>	Where CSS can load from
<code>img-src</code>	Where images can load from
<code>connect-src</code>	Where fetch/XHR calls can go
<code>default-src</code>	Fallback for all unspecified directives

Report-Only Mode

Content-Security-Policy-Report-Only: script-src 'self'

- Violations reported but not blocked
- Use when rolling out CSP to avoid breaking the site

Industry Examples

- **GitHub** — strict CSP blocks all inline scripts; enforces `script-src 'self'`
 - **Google** — uses nonces to allow specific inline scripts safely
 - **Banks / payment pages** — CSP required by PCI-DSS to prevent card skimming
-

Interview Questions

Q: What is CSP and what attack does it prevent?

CSP is an HTTP response header that controls which content sources the browser trusts. It prevents XSS — Cross-Site Scripting attacks — where attackers inject malicious scripts into a page. Even if a script is injected, the browser blocks it if the source isn't whitelisted.

Q: How do you allow scripts only from your own domain?

Set `Content-Security-Policy: script-src 'self'` — browser will only execute scripts from the same origin.

Q: How do you safely allow a specific inline script with CSP?

Use a nonce — a unique token generated per request. Add it to the script tag and the CSP header. The browser allows that specific script and blocks all others.

Q: How do you test CSP without breaking your site?

Use `Content-Security-Policy-Report-Only` — violations are logged but not enforced. Roll out in report-only mode first, fix violations, then switch to enforcing mode.